

Neeraj — Full-Stack Personal Portfolio

Complete Operational Manual: Access Links, Administration, Multi-System Setup, Credential Management & GitHub Cloud Hosting

ANGULAR 21 STANDALONE

JAVA 22 + SPRING BOOT 3.3

SPRING SECURITY 6 (JWT)

POSTGRESQL 16 / H2 DEV

DOCKER MULTI-STAGE

 Document Contents

- 1. Master Access Links & URLs (Public Website, Admin Portal, REST API)
- 2. How to Open the Website & Admin Profile (Navigation, Default Credentials, 1-Click Filler)
- 3. How to Launch on Another Computer (Windows, macOS, Linux — Local Dev vs Docker)
- 4. System Architecture & Functionality Overview (Frontend, Backend, Database, Features)
- 5. How to Change Admin Credentials (Java Code, Database SQL, BCrypt Hashing)
- 6. How to Host on GitHub & Cloud Deployment (Git Push, Static vs Dynamic, Free Hosting Options)

1. Master Access Links & Quick Reference

When running locally on your computer, the application operates through the following primary URLs:

Service / Mode	URL Link	Description
Public Visitor Website	http://localhost:4200	Main portfolio website displaying projects, photo gallery with lightbox, expeditions, articles, skills, and contact form.
Admin Portal Login	http://localhost:4200/admin/login	Secured authentication portal for the administrator to sign in with email and password.
Admin Dashboard (Command Center)	http://localhost:4200/admin/dashboard	Overview of published content statistics, photo vault metrics, and contact inquiry alerts.
Backend REST API Base	http://localhost:8080/api	Spring Boot REST API service handling database persistence, authentication, and file uploads.
Docker Production URL	http://localhost (Port 80)	When launched using Docker Compose, the complete stack is unified on standard HTTP port 80.

Key Public REST API Endpoints

Method	Endpoint	Sample Use Case
GET	<code>/api/public/projects</code>	Returns list of all published software engineering case studies.

Method	Endpoint	Sample Use Case
GET	/api/public/photos	Returns all photography vault photos including camera EXIF parameters.
GET	/api/public/albums	Returns photography albums (e.g. Kanchenjunga, Karakoram, Western Ghats).
GET	/api/public/travel	Returns high-altitude expedition logs and trek notes.
GET	/api/public/blog	Returns technical articles and mountaineering field guides.
GET	/api/public/skills	Returns skills categorized by Frontend, Backend, Database, Cloud, and Photography.
POST	/api/public/contact	Accepts name, email, subject, and message from visitors and notifies the admin.

2. How to Open the Website & Admin Profile

A. Public Visitor Experience

To view your website as any public visitor, open your browser and go to: <http://localhost:4200>

- **Home Page:** Hero section with animated GSAP role transitions (*"Software Developer"*, *"Himalayan Photographer"*, *"Alpine Mountaineer"*), metrics ribbon, featured projects, photography masonry preview, high-altitude expeditions, and contact CTA.
- **Projects Showcase** (`/projects`): Interactive filter pills by technology and deep-dive architecture case studies (`/projects/:slug`).
- **Photography Gallery** (`/photography`): Album categories with a **fullscreen Lightbox** showing camera EXIF data (Camera Body, Lens, Focal Length, Aperture, Shutter Speed, ISO, Location).
- **Expedition Journals** (`/travel`): Route elevation profiles, trek duration, technical difficulty, and field photography galleries.
- **Field Articles** (`/blog`): Markdown-rendered engineering articles and high-altitude gear guides.
- **Skills Matrix** (`/skills`): Grouped interactive matrix with proficiency meters.
- **Contact Section** (`/contact`): Interactive message submission form with live validation and toast alerts.

B. Admin Profile & Content Management Mode

To access the administrative dashboard, navigate to: <http://localhost:4200/admin/login>

Default Admin Credentials

Email: `admin@portfolio.com`

Password: `admin123`

Shortcut: You can also simply click the **"Fill Demo Credentials"** button on the login screen!

Admin Management Modules (Available in Left Sidebar):

- **Dashboard** (`/admin/dashboard`): Live counts of all projects, photos, albums, expeditions, articles, skills, and unread contact messages.
- **Projects Manager** (`/admin/projects`): Create new projects, edit architectural notes, update GitHub/live demo URLs, and toggle **Draft / Published** status with a single click.
- **Photo Vault** (`/admin/photos`): Upload high-resolution images, input camera EXIF metadata, select album category, and manage publication.
- **Albums Manager** (`/admin/albums`): Organize photos into themed albums (Mountains, Nature, Travel, Architecture).
- **Expedition Logs** (`/admin/travel`): Log new Himalayan treks with altitude (e.g. 5,319m), difficulty tags, and gear checklists.
- **Articles** (`/admin/blogs`): Author technical articles with estimated reading times, tag pills, and publication states.
- **Skills Matrix** (`/admin/skills`): Add new skills with category selection and proficiency percentage sliders (0-100%).

- **Contact Inbox** (`/admin/messages`): Review incoming visitor inquiries, mark messages as read, and track replied status.

3. How to Launch on Another Computer / System

When you transfer this project to another laptop, desktop, or server (Windows, macOS, or Linux), follow this guide.

Prerequisites to Install on the New System

- **Java Development Kit (JDK)**: Version 21 or 22 (e.g., Eclipse Temurin or Oracle JDK).
- **Apache Maven**: Version 3.9+ (or use the built-in Maven command).
- **Node.js**: Version 20+ or 22+ (includes `npm`).
- **Docker & Docker Desktop** (Optional, only needed if you prefer running inside containers).

Method 1: Instant Local Development Launch (Zero Setup)

The backend includes an automatic `dev` profile with an in-memory database. You do **not** need to install or configure PostgreSQL to run locally!

Step 1: Start the Backend API (Terminal 1)

```
cd portfolio/backend
mvn spring-boot:run
```

Spring Boot will start on **port 8080**, create database tables automatically, and seed all projects, albums, photos, expeditions, and the admin account.

Step 2: Start the Frontend Application (Terminal 2)

```
cd portfolio/frontend
npm install
npm start
```

Angular dev server will compile and start on **port 4200**, proxying all `/api` and `/uploads` calls to `localhost:8080`.

Step 3: Open in Browser

Open `http://localhost:4200` for the visitor site, or `http://localhost:4200/admin/login` for the admin portal.

Method 2: Docker Compose Launch (Production Mode)

If the new system has Docker Desktop installed, you can launch PostgreSQL, the Spring Boot backend, and the Angular frontend in containers with a single command:

```
cd portfolio
docker-compose up --build
```

Container: portfolio-frontend

Container: portfolio-backend

Serves the Angular production build via Nginx on **port 80** (`http://localhost`). Reverse proxies API calls to backend.

Runs Spring Boot Java 22 on **port 8080** connected to containerized PostgreSQL.

Container: **portfolio-postgres**

Runs PostgreSQL 16 on **port 5432** with persistent volume `postgres_data` .

Volume: **portfolio_uploads**

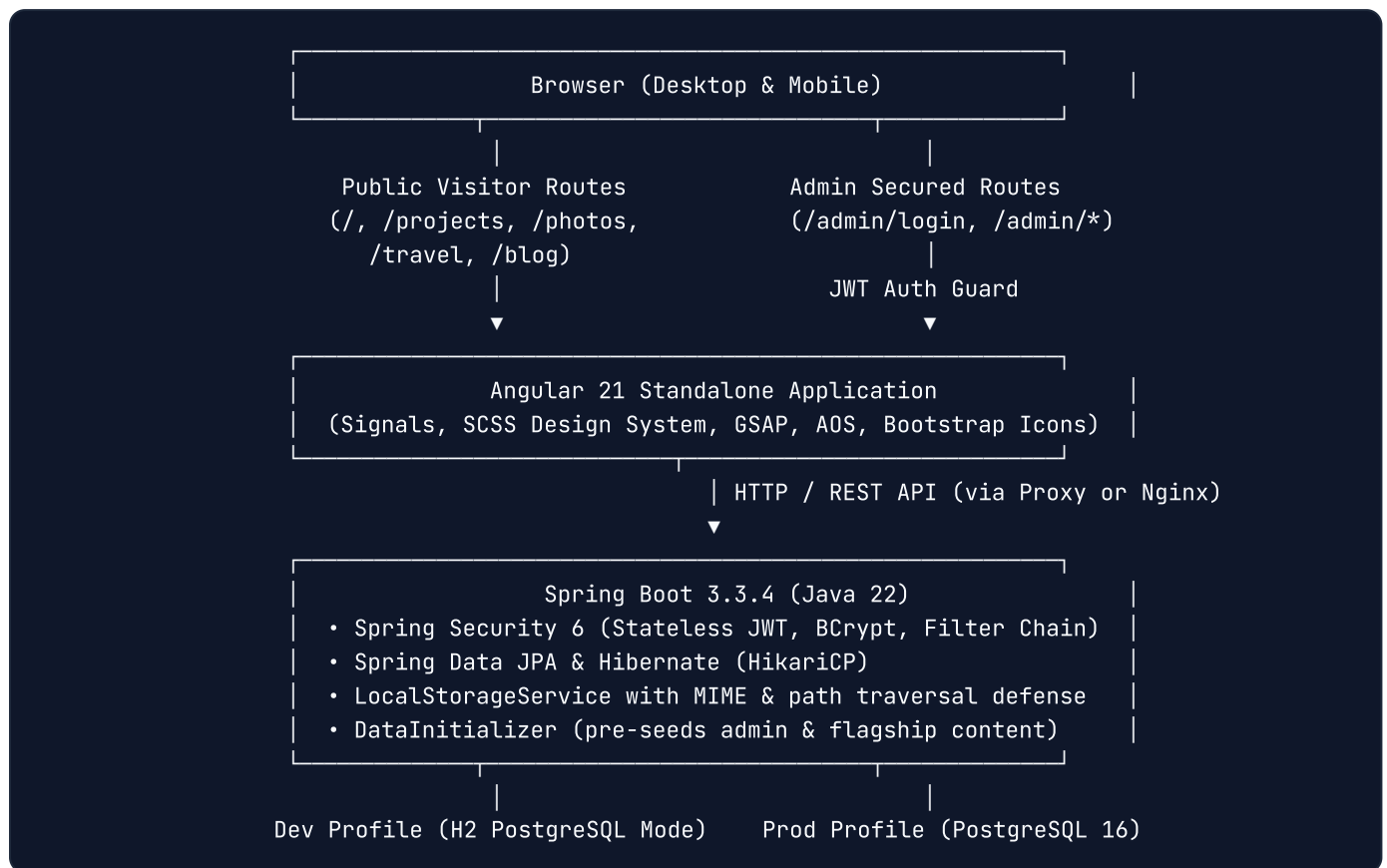
Persists uploaded photo vault images across container restarts.

⚠ Troubleshooting Port Conflicts

If port 8080 or 4200 is already used by another application on your system:

- Backend: Change `server.port=8081` in `backend/src/main/resources/application.properties` and update `target` in `frontend/proxy.conf.json` .
- Frontend: Run `ng serve --port 4201` .

4. System Architecture & How It Works



Core Functionalities Explained

- **Dual Security Layers:**
 - **Frontend Guard** (`auth.guard.ts`): Verifies JWT validity in browser memory/storage. Redirects unauthorized visitors to `/admin/login` with `returnUrl` preserving their intended page.
 - **Backend Security Filter** (`JwtAuthenticationFilter.java`): Validates the signed HMAC SHA-256 Bearer token on every administrative HTTP request. Rejects expired or forged tokens with HTTP 401/403.
- **Publishing Workflow:** Public endpoints query only records where `published = true`. The admin portal allows editing drafts privately before flipping them live with a single click.
- **Image Storage Architecture:** Uploaded files are validated against MIME type (JPEG, PNG, WebP), checked against path traversal attacks (`..`), renamed with timestamped UUIDs, and served through a protected static handler at `/uploads/**`.
- **EXIF Camera Extraction:** Photos maintain complete technical camera data (lens, focal length, aperture, shutter speed, ISO) displayed dynamically in the public lightbox.

5. How to Change Admin Credentials

You can change the administrator email and password through any of the following methods:

Method A: Change Default Credentials in Java Code (Recommended for Dev)

Open the file: `backend/src/main/java/com/neeraj/portfolio/service/DataInitializer.java`

Locate lines 47–56 in the `seedUsers()` method:

```
User admin = new User();
admin.setUsername("admin");
// 🛠️ Change your email here:
admin.setEmail("your-new-email@domain.com");
// 🛠️ Change your password here (BCrypt automatically hashes it):
admin.setPasswordHash(passwordEncoder.encode("YourNewStrongPassword2026!"));
admin.setFullName("Neeraj");
admin.setRole(User.Role.ROLE_ADMIN);
admin.setEnabled(true);
userRepository.save(admin);
```

Save the file and restart the Spring Boot backend. The new credentials will take effect immediately.

Method B: Update Password in PostgreSQL Database via SQL

If you are running in PostgreSQL mode and want to change the password directly in the database without restarting the code:

1. Generate a BCrypt hash for your new password (using an online tool like bcrypt-generator.com with 12 rounds, or via Java).
2. Connect to PostgreSQL and execute:

```
-- Connect to your portfolio database
\c portfolio_db;

-- Update admin email and password hash:
UPDATE users
SET email = 'your-new-email@domain.com',
    password_hash = '$2a$12$YourGeneratedBcryptHashStringHere'
WHERE username = 'admin';
```

Method C: Update JWT Secret Key (For Production Security)

For enhanced production security, change the JWT secret key in

`backend/src/main/resources/application.properties`:

```
app.jwt.secret=YourCustom64CharacterOrLongerHexadecimalOrBase64SecretKeyHere
app.jwt.expiration-ms=86400000 # 24 hours in milliseconds
```

6. How to Host on GitHub & Cloud Deployment Guide

Part 1: Pushing Your Entire Project to GitHub

Follow these steps to store and version control your portfolio on GitHub:

Step 1: Open your terminal at the root of the project (`portfolio/`):

```
git init
```

Step 2: Check that `.gitignore` is present

The project already includes a production-ready `.gitignore` file that automatically excludes bulky and sensitive files:

- Frontend: `node_modules/`, `dist/`, `.angular/`
- Backend: `target/`, `.class` files, temporary logs
- Local storage uploads: `uploads/*`

Step 3: Stage and commit all files:

```
git add .  
git commit -m "feat: complete modern full-stack personal portfolio with admin CMS"
```

Step 4: Create a new repository on GitHub:

1. Go to <https://github.com/new> in your browser.
2. Name your repository (e.g., `portfolio` or `neeraj-portfolio`).
3. Choose **Public** (or Private). Do *not* check "Initialize with README" since you already have files.
4. Click **Create repository**.

Step 5: Link and push to GitHub:

```
git branch -M main  
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/portfolio.git  
git push -u origin main
```

Part 2: Important Note on GitHub Pages vs Full-Stack Apps

Understanding GitHub Pages Limitations

GitHub Pages is strictly for static websites (HTML/CSS/JS only).

Because your portfolio is a true full-stack application featuring a live **Spring Boot Java API**, **JWT authentication**, **photo uploads**, and **database persistence**, GitHub Pages alone cannot run the backend. To host your entire application live on the internet, follow the recommended cloud hosting options below.

Part 3: Recommended Free / Low-Cost Hosting Deployment

Option A: Decoupled Cloud Hosting (Best Free Tier)

1. Database: Neon / Supabase (Free)

Create a free cloud PostgreSQL database at neon.tech or supabase.com. Copy the JDBC connection URL.

2. Backend: Render / Railway (Free/Cheap)

Deploy `backend/Dockerfile` to render.com or railway.app. Set environment variables for DB credentials.

3. Frontend: Vercel / Netlify (Free)

Connect your GitHub repository to vercel.com or Netlify. Set build command to `npm run build` and output to `dist/frontend/browser`.

4. Custom Domain

Point your personal custom domain (e.g., `neeraj.dev`) to Vercel/Netlify for free SSL HTTPS.

Option B: Single VPS Server with Docker (Most Professional, ~\$5/month)

Rent a cheap Linux cloud server (DigitalOcean Droplet, Linode, or AWS EC2 t3.micro):

```
# 1. Install Docker & Docker Compose on your cloud server
# 2. Clone your GitHub repository:
git clone https://github.com/YOUR_GITHUB_USERNAME/portfolio.git
cd portfolio

# 3. Launch with Docker Compose in detached mode:
docker-compose up -d --build
```

Your entire full-stack application (PostgreSQL + Spring Boot API + Angular Nginx) will be running live 24/7 on your server!